Project Activity 2 Report: Baseline CNN for Fine-Grained Vehicle Classification

Introduction to AI Programming, Fall 2026 — Project 1, Activity 2
Team: 20230906 Heeseon Yoo 20230613 Donghyun Shin 20230673 Seojin Han 20230497 Hyeonil Cho
Dataset: `https://github.com/faezetta/VMMRdb` Code: `github.com/heeseon28/intro_ai_project1/tree/main/task2`

1. Dataset and Data Preparation

We use the 50-class subset of VMMRdb [1] fixed in Activity 1: the 5 largest make-model classes of 10 makes (production years merged), i.e. a **50-way fine-grained make-model classification** task.

- **Splits**: 118,471 images after removing exact duplicates, split once with per-class stratification into **82,929 train / 17,771 validation / 17,771 test** (70/15/15). Identical (MD5) images never cross splits. The split CSVs are versioned with SHA-256 hashes so that all later experiments reuse them. The **test set is reserved** and was not used.
- **Imbalance**: training images per class range from 184 (Jeep Patriot) to 4,805 (Honda Civic), a 26× gap.
- **Preprocessing** (train and validation identically): RGB conversion, resize to 224×224, tensor conversion, normalization with mean = std = 0.5 per channel. No augmentation.

2. Model and Training Configuration

Table 1: Baseline configuration (recorded in `config.json`).

Architecture	ResNet-18 [2] (torchvision), FC 512→50, 11.2M params
Initialization	random (`weights=None`), no pre-training
Image size	224×224 RGB
Batch size / epochs	64 / 12 (fixed, no early stopping)
Optimizer	Adam, learning rate 3×10^{-4}, no weight decay, no LR schedule
Loss	standard (unweighted) cross-entropy
Not used	augmentation, transfer learning, re-weighting/sampling, special losses
Checkpoint	epoch with the highest validation macro-F1
Reproducibility	seed 0, FP32, deterministic cuDNN (bit-exact only on the same hardware)
Hardware	run A: GPU slice, 3.1 min/epoch; run B: CPU, 106 min/epoch

3. Training and Validation Performance

Because the classes are imbalanced, we report **macro-F1** (mean of per-class F1) and balanced accuracy (mean per-class recall) in addition to accuracy. Table 2 re-evaluates the selected checkpoint on both splits. To check that the baseline is a reliable reference, the same code, split and seed were run twice by different members on different hardware (run A: GPU, run B: CPU).

Table 2: Performance of the selected checkpoint. Run A (epoch 12) is the reported baseline; run B (epoch 10) is an independent re-run.

Run	Split	Loss	Accuracy	Macro-F1	Bal. acc.
A	Train	0.109	96.4%	95.6%	95.7%
A	Validation	0.984	**77.0%**	**73.5%**	72.4%
B	Train	0.136	95.5%	94.5%	94.5%
B	Validation	0.920	77.1%	73.2%	72.8%

Q1: How well does the baseline classify the dataset? About three of four validation images are classified correctly (chance: 2%), but macro-F1 is 3.6 pp below accuracy and per-class F1 ranges from 0.39 to 0.93 (median 0.77): performance is uneven across classes. The result is **reproducible**: the two runs differ by 0.1 pp in accuracy and 0.2 pp in macro-F1, and their per-class F1 scores correlate at 0.92, so this baseline is a stable reference for later comparisons.

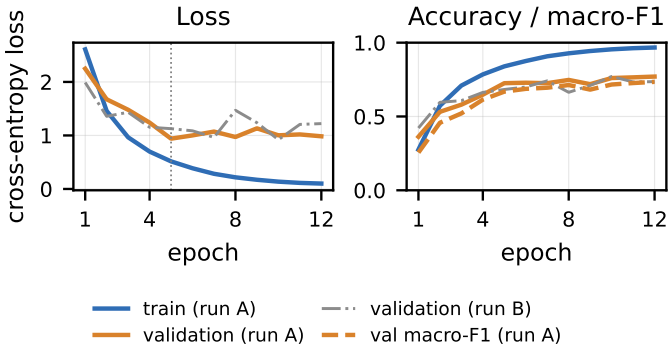


Figure 1: Learning curves of run A, with run B's validation curves (grey). Train values are averaged during each epoch's updates; the dotted line marks run A's minimum validation loss (epoch 5).

Overfitting, not underfitting. Training accuracy reaches 96.4% while validation accuracy is 77.0% (**19.3 pp gap**; run B: 18.4 pp). In run A the validation loss stops improving after epoch 5 (0.94) and then oscillates between 0.94 and 1.13, while the training loss keeps falling to 0.10 (Fig. 1). The model fits the training set far better than unseen images; it is not underfitting, since training accuracy is near 100%. Validation performance is also **unstable without an LR schedule**: in run B, validation accuracy drops from 74.1% to 66.4% between epochs 7 and 8, and from 77.1% (epoch 10) to 73.6% at the last epoch. Longer training alone is therefore unlikely to help, and the checkpoint must be chosen on the validation set. This choice makes the validation numbers slightly optimistic, which is why the untouched test set is kept for the final, unbiased comparison.

4. Confusion Matrix and Difficult Classes

Table 3: Lowest-F1 classes, their training size, and their most frequent wrong prediction (% of the class).

Class	n_{train}	F1	Most confused with
Dodge Ram 1500	1,046	0.39	Ram 2500 (33%)
GMC Sierra 1500	1,124	0.42	Chevy Silverado (27%)
BMW 328i	594	0.45	BMW 325i (19%)
Dodge Ram 2500	867	0.45	Ram 1500 (23%)
BMW 528i	319	0.50	BMW 530i (16%)
VW Golf	330	0.52	VW Jetta (9%)
Jeep Cherokee	775	0.53	Grand Cherokee (16%)

Q2: Which classes or images are difficult? (Fig. 2, Table 3, Fig. 3). The same classes are hardest in both runs (328i, Ram 1500, Sierra, Golf, Cherokee and Ram 2500 are in both bottom-six lists).

- **Look-alike pairs dominate the hardest classes**, as predicted in Activity 1: badge-engineered twins (Sierra→Silverado 27%), size variants of one truck (Ram 1500↔2500), and trims of one series (328i→325i 19%, GTI→Golf 19%). The largest single error count is Civic→Accord (133 images), two similar sedans of the same make.

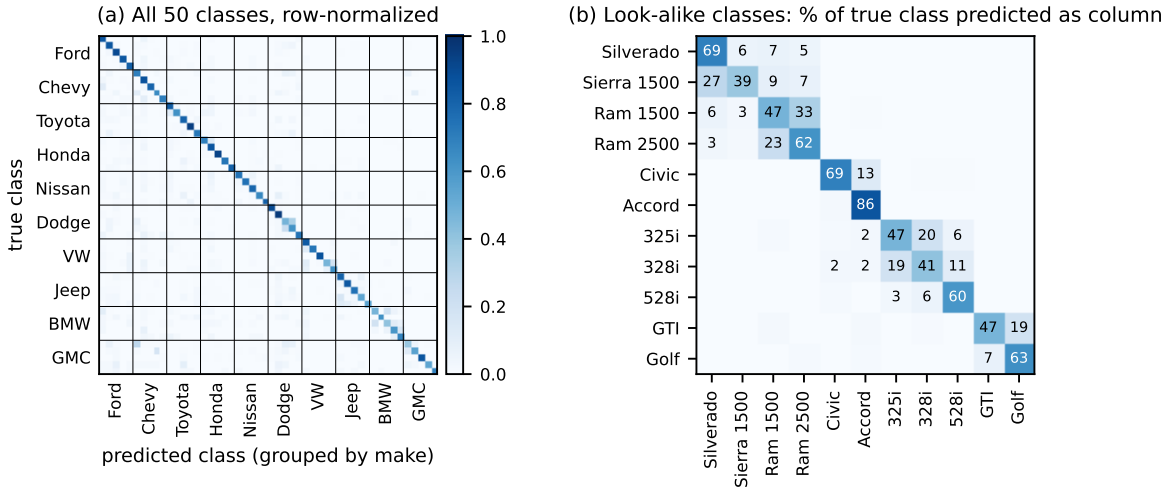


Figure 2: Validation confusion matrix. (a) All classes, ordered by make; off-diagonal mass is spread across makes. (b) Look-alike classes; each number is the % of a true class (row) predicted as the column class.



Figure 3: Misclassified validation images for the main confusion pairs (first error of each pair in file order, not hand-picked).

- **Most errors cross makes**: of the 23.0% validation error, **16.8 pp (73%) go to a different make** and only 6.1 pp stay within the correct make. Typical cross-make errors share a body type (Silverado→Ram 1500, Corvette→Mustang, Altima→Accord). This suggests that the scratch-trained CNN relies mainly on the global silhouette rather than on small make cues such as emblems and grilles (to be checked with Grad-CAM in the next activity).
- **Visual distinctiveness matters more than class size.** Larger classes tend to score higher (correlation of $\log n_{\text{train}}$ with F1: 0.44), and small classes are weak (528i 0.50, Golf 0.52, Patriot 0.65). But the two worst classes have over 1,000 training images, while the best ones have unique shapes (Odyssey 0.93, Grand Caravan 0.92, Beetle 0.91).
- **Difficult images: old and rare generations.** Merging years mixes several generations in one class. Validation accuracy is 59.5% for cars built before 1990, 72.3% for the 1990s, and 81.1% for the 2000s (the dominant decade), but drops to 68.2% from 2010. Classic cars are often mistaken for other classics (1960s Mustang→Impala, 1963 Corvette→Mustang; Fig. 3).

5. Directions for Improvement

Q3: What should we investigate next? Each change will be tested on the same splits, with the same metrics, and with one change at a time against this baseline.

1. **Reduce overfitting** (19.3 pp gap): data augmentation (random crop, flip, colour jitter), weight decay, and a learning-rate schedule with early stopping on validation macro-F1.
2. **Transfer learning**: ImageNet-pre-trained weights should provide generic shape and part features that 82k images cannot teach from scratch. As a reference upper bound, a separate run on the same splits with pre-training, augmentation, AdamW and a cosine schedule reached $94.2 \pm 0.1\%$ validation accuracy over 3 seeds. Because it changes several factors at once, it does not isolate the effect of pre-training; the controlled one-change experiments above will.
3. **Use the class hierarchy** (our Activity 1 hypotheses): since 73% of errors cross makes, a make- or body-type-level first stage could remove many of them in the scratch setting. We will check whether this gain survives with pre-training.
4. **Target look-alike pairs**: aspect-preserving resize and a larger input size to keep emblem and grille detail. Class-balanced loss [3] for the small classes (Patriot, 528i, Golf).
5. **Reliability**: repeat each setting with 3 seeds and report mean ± std (runs A and B used the same seed, so they measure hardware/run variation, not seed variation). The test set is used only for the final comparison.

6. Code and Reproduction

`bash task2/run_task2.sh --data-root <VMMRdb> --device cuda` trains and evaluates the baseline (`task2/train_baseline.py`) and writes the configuration with split and source hashes, the epoch history, metrics, confusion matrix, per-class scores and predictions to `result_task2/<run>/`. The submitted archive contains the code, split files, both result folders (run A: `baseline_seed0_DonghyunShin`, run B: `baseline_seed0_Hyeonil_Cho`) and a `README.md` with setup and run instructions. *Limitations*: one seed; near-duplicate images were not audited. Run B's recorded hashes differ from run A's only because Windows converted line endings (contents verified identical).

References

[1] F. Tafazzoli, H. Frigui, K. Nishiyama, "A Large and Diverse Dataset for Improved Vehicle Make and Model Recognition," *CVPR Workshops*, 2017.

[2] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016.

[3] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, S. Belongie, "Class-Balanced Loss Based on Effective Number of Samples," *CVPR*, 2019.